

第四章 软件设计 选择题解析

对应 第四章_软件设计_选择题.md，每题附详细解析与坑点说明。

第 1 题

答案：B

解析：

这题考的是总体设计与详细设计的阶段边界——总体设计管"结构"，详细设计管"过程"。

工程师在总体设计阶段就写了快速排序的详细伪代码和循环结构，这是把"模块内部怎么实现"提前到了"模块怎么划分"阶段。总体设计只负责把系统分解成模块、确定模块间接口，不碰模块内部的算法逻辑。

• A错：总体设计不该写算法细节，这正是越界点。

• B正确：越界到详细设计。总体设计划模块、定接口；详细设计写算法、定逻辑。

• C错：伪代码不是编码，编码要写真正的源代码。

• D错：与需求分析无关。

口诀：总体划模块，详细写算法。总体管"结构"（模块划分+接口），详细管"过程"（模块内部逻辑）。

第 2 题

答案：B

解析：

这题与第1题互补，考的还是阶段边界，但方向反了——详细设计阶段去定了模块间接口。

确定模块A调用模块B的接口参数，这是"定模块间关系"，属于总体设计的任务。详细设计是在接口已经确定的前提下，设计模块内部的算法逻辑。

• A错：详细设计不定模块间接口，接口是总体设计定的。

• B正确：越界到总体设计。详细设计的前提是接口已定，它只管填模块内部。

• C、D错：与编码、测试无关。

与第1题对照记忆：总体设计=划模块+定接口，详细设计=写算法+定内部逻辑。谁干了谁的活，就是越界。

第 3 题

答案：D

内聚7级从低到高：偶然→逻辑→时间→过程→通信→顺序→功能。功能内聚是最高——模块内所有成分共同完成一个单一功能，是最理想状态。口诀"偶逻时过通顺功"，最后一个"功"是最高。

第 4 题

答案：A



偶然内聚是最低——模块内各成分毫无关系，只是碰巧放在一起。口诀"偶选时过通顺功"，第一个"偶"是最低。注意"最低"和"最高"不要记反。

第 5 题

答案：D

非直接耦合是最低（最好）——模块间无直接联系，通过主模块间接控制。常见错误选A"数据耦合"：数据耦合确实是好的耦合，但理论上非直接耦合更低。以张海藩教材为准，部分教材口径不同。口诀"内公外控特数非"，最后一个"非"是最低。

第 6 题

答案：C

内容耦合是最高（最差）——一个模块直接访问另一个模块的内部内容，应完全避免。口诀"内公外控特数非"，第一个"内"是最高。

第 7 题

答案：C

解析：

这题考的是三种耦合类型的区分依据——看传递的是什么。

拆解判断依据：

- ①传学号（int）：传的是简单数据参数 → 数据耦合
- ②传完整Student结构体但只用学号：传了"整个数据结构但只用一部分" → 特征耦合
- ③传flag决定执行哪段代码：传的是控制信息 → 控制耦合
- A错：②③不是数据耦合。②传整个结构只用部分是特征耦合，③传控制信息是控制耦合。
- B错：①②搞反了。①传简单数据是数据耦合不是特征耦合。
- C正确：数据耦合、特征耦合、控制耦合。
- D错：②③搞反了。

判断耦合类型的三步法：看传什么→是数据、整个结构、还是控制信息→对应数据耦合、特征耦合、控制耦合。数据耦合传"够用的简单数据"，特征耦合"传多了"，控制耦合传"指挥别人怎么做的flag"。

第 8 题

答案：B

解析：

这题考的是控制耦合与数据耦合的本质区别——不在参数的"形式"，而在参数的"语义"。

flag确实以数据形式传递（它是个int），但这不是关键。关键是flag的语义：它不是被B使用的数据，而是A用来"指挥B执行哪段代码"的控制信息。flag决定了B的执行路径——这才是控制耦合的本质。

- A错（强干扰项）：flag"也是数据"这个说法在形式上没错，但混淆了"数据形式"和"数据语义"。数据耦合传递的数据是被调用者使用的，控制耦合传递的flag是被调用者据以选择执行路径的。flag不参与计算，只参与"走哪个分支"——这就是控制信息。
- B正确：控制耦合。flag的语义是控制B执行哪段代码，比数据耦合更危险（因为B的行为依赖A的指



挥，耦合度更高)。

• C错：特征耦合是传整个数据结构只用一部分，这里没传结构。

• D错：公共耦合是共享全局数据，这里是通过参数传递。

区分数据耦合和控制耦合的关键：问“这个参数是给B用的，还是给B指路的？”给B用的是数据耦合，给B指路（决定走哪个分支）的是控制耦合。flag、开关、状态码这类“指挥棒”参数，无论它是什么类型，都是控制耦合。

第9题

答案：B

解析：

这题考的是特征耦合与数据耦合的区分——传“正好够用的”还是传“整个结构但只用一部分”。

方式1：传整个Student结构体，B只用了学号和姓名——传了不需要的字段（比如还有年龄、地址等B根本不用的）。这就是特征耦合（也叫标记耦合），耦合度比数据耦合高，因为B暴露在了它不需要的数据结构面前。

方式2：只传学号和姓名——传的正好是B需要的。这是数据耦合，耦合度更低，是更好的设计。

• A错：方式1不是数据耦合（传整个结构只用部分是特征耦合），“方便”不等于“好设计”。

• B正确：方式2更好（数据耦合），方式1是特征耦合（传多了）。

• C错：方式1才是特征耦合，方式2是数据耦合，搞反了。

• D错：方式2只传需要的简单参数，是数据耦合不是特征耦合。

口诀“数据正好够，特征给多了”。设计原则：只传调用者需要的，不要把整个结构甩过去。传多了，被调用者就暴露在了它不该知道的数据面前——一旦结构体改了，所有传整个结构的模块都要跟着改。

第10题

答案：C

模块设计的黄金法则：内聚越高越好（模块内部紧密结合），耦合越低越好（模块之间松散联系）。两者方向相反。口诀“内聚高，耦合低”。

第11题

答案：B

解析：

这题考的是扇出扇入的设计原则——扇出适中，扇入越大越好。

扇出是一个模块直接调用的下级模块数。A扇出7，远超推荐的3-4个——这意味着A管得太宽，职责过重，修改A会影响7个下级。改进方法是增加中间层，让A只管2-3个中层模块，再由中层管理下级。

扇入是调用一个模块的上级模块数。B扇入1（只被A调用），说明B的复用性低。扇入越大越好——被越多模块调用说明越通用。

• A错：B扇入小不是当前的主要问题，A扇出过大才是。

• B正确：A扇出过大应分层，扇入越大越好。

• C错：扇出7过大，设计有问题。

• D错：应增加扇入（提高复用），不是减少。



扇出是"向下管几个孩子" (别太多, 3-4个为宜, 否则管不过来); 扇入是"向上几个爹叫你" (越多越好, 说明你越通用)。扇出过大→加中间层分流; 扇入太小→提炼通用模块。

第 12 题

答案: C

解析:

这题考的是McCabe环形复杂度的计算及其与测试用例数的关系。

分两步:

- 第一步算 $V(G)$: $V(G) = E - N + 2 = 10 - 8 + 2 = 4$
- 第二步关联测试: $V(G)$ 的物理意义是程序的独立路径数, 基本路径法要求至少 $V(G)$ 个测试用例才能覆盖所有独立路径。所以至少需要4个测试用例。
- A错: $V(G)$ 算对了 ($E - N + 2 = 4$) 但漏了"独立路径数= $V(G)$ "这个关联, 误以为用例数与 $V(G)$ 无关。
- B错 (强干扰项): $V(G) = 4$ 算对了, 但"至少2个用例"错了。 $V(G)$ 就是独立路径数, 至少要4个用例。

。

- C正确: $V(G) = 4$, 至少4个测试用例。
- D错: 公式用错 ($E - N + 1 = 3$), 少加了1。

两个关键点一次记牢: ①公式"边减点加二" ($E - N + 2$); ② $V(G)$ =独立路径数=基本路径法最少测试用例数。这两个是等价的—— $V(G)$ 不只是个图论数字, 它直接告诉你"至少要写几个测试用例"。

第 13 题

答案: C

解析:

这题考的是结构程序设计的核心——只用三种基本控制结构, 禁止goto。

三种基本控制结构是: 顺序、选择 (if)、循环 (while/for)。if和while都是允许的。代码的问题在于使用了goto跳转——goto从if块跳入while循环内部 (标签L处), 破坏了程序整体的单入口单出口原则, 导致控制流不可预测。

• A错 (强干扰项): 说goto"破坏了if块的单入口单出口"——听起来有道理, 但goto破坏的是程序整体的控制流, 不是if块本身。if块本身的入口出口是正常的, 问题是goto让控制流从if块跳到了while内部, 跨越了正常结构边界。

• B错 (强干扰项): 说"if和while结构过多导致混乱"——结构程序设计不限制控制结构的数量, 限制的是goto跳转。用100个if也没问题, 但用1个goto就违反了结构化。

• C正确: goto跳转破坏了程序整体的单入口单出口原则。

• D错: for不是必需的, while就够了。

结构程序设计=顺选循+不用goto。goto的问题在于它让控制流可以随意跳跃, 破坏每个模块"一个入口、一个出口"的规整结构。注意: 破坏的是整体控制流, 不是某个具体结构 (如if或while) 本身的规整性——if和while自身都是合法的基本结构。

第 14 题

答案: A

解析:

这题考的是独立路径数与所有可执行路径的区别—— $V(G)$ 是下限, 不是全部。



$V(G)=3$ 意味着有3条独立路径，基本路径法覆盖这3条就达标了。但"独立路径"不等于"所有可执行路径"——循环每多执行一次，就产生一条新的执行轨迹（虽然它可能不引入新的独立路径）。所以即使3条独立路径全覆盖了，循环执行1次、2次、3次的情况仍然不同，可能暴露不同的错误。

- A正确：独立路径数 $\neq$ 所有可执行路径数。循环执行次数变化会产生新的执行情况，基本路径法不保证覆盖这些。
- B错： $V(G)$ 没算错，问题在于独立路径本身就不等于所有路径。
- C错：语句覆盖比基本路径法弱，不是改进方向。
- D错：基本路径法有效，但有其局限性——它保证独立路径覆盖，不保证所有执行情况覆盖。

$V(G)$ 是"独立路径数"的下限，不是"所有可执行路径数"。有循环时，可执行路径数可以无限（循环执行次数不定），但独立路径数是有限的（ $V(G)$ ）。基本路径法保证独立路径覆盖，这是它的价值；但循环情况下的多次执行需要其他方法（如边界值分析循环次数）补充。这揭示了白盒测试的一个本质局限：覆盖了所有独立路径，仍然不能保证没有错误。

第 15 题

答案：C

解析：

这题考的是通信内聚与顺序内聚的区分——关键是"共享数据"还是"数据流传递"。

模块内两个操作：①排序 ②统计前10名平均分。两者都作用于同一组成绩数据。

顺序内聚的要求是"前一步的输出是后一步的输入"——有数据流传递。但这里排序的结果（排好序的数组）不是统计的"输入"，统计用的是"同一组成绩数据"，两个操作是平级的、共享同一数据集的操作。这是通信内聚的标志。

- A错（强干扰项）：看似"先排序再统计"有顺序，但顺序内聚要求前一步输出是后一步输入（数据流），而这里是两个操作共享同一数据集（不是输出 $\rightarrow$ 输入的关系）。区分关键：数据流传递=顺序内聚，共享数据集=通信内聚。
- B错：过程内聚强调"按步骤执行"但无数据关系，不够精确——本题有明确的共享数据。
- C正确：通信内聚，操作同一数据集。
- D错：不是单一功能，是两个不同操作。

顺序内聚vs通信内聚是高频混淆点。判断法：前一步的输出变成后一步的输入（像流水线） $\rightarrow$ 顺序内聚；两步操作同一组数据（像两个工人处理同一堆零件） $\rightarrow$ 通信内聚。顺序内聚（第6级）高于通信内聚（第5级）。

第 16 题

答案：A

解析：

这题考的是偶然内聚与逻辑内聚的区分——有没有"逻辑相似性"和"选择机制"。

- 模块甲："计算平均成绩"和"发送通知邮件"——这两个功能毫无关系，既不相似也没有flag选择。纯粹碰巧放在一起 $\rightarrow$ 偶然内聚。
- 模块乙："计算平均成绩"和"找出最高分"——这两个功能逻辑相似（都是统计计算），且有flag选择执行哪个 $\rightarrow$ 逻辑内聚。
- A正确：甲偶然（无关系），乙逻辑（相似+flag）。
- B错：甲不是逻辑内聚（"计算平均"和"发邮件"无逻辑相似性），乙不是偶然内聚（有flag选择）。
- C错：乙有逻辑相似性和flag选择，是逻辑内聚不是偶然内聚。



- D错：甲无逻辑相似性，是偶然内聚。
- 偶然内聚vs逻辑内聚的区分：偶然内聚=各成分"毫无关系"地放一起；逻辑内聚=各成分"逻辑相似"且用flag选择。有没有相似性、有没有选择机制，是判断依据。

第 17 题

答案：C

解析：

这题考的是信息隐藏原理——模块内部信息应对外不可见，只能通过接口访问。

模块A没有通过B的接口函数获取数据，而是直接伸手进B的内部数组data[]自行遍历——这等于B的内部数据结构对A完全暴露。一旦B的data[]改了实现（比如换成链表），A的代码也会崩。这正是信息隐藏原理要避免的。

- A错：模块化是"把系统分解成模块"，不是这里的暴露问题。
- B错（强干扰项）：抽象是"抽出本质特征隐藏细节"，方向看似对但不精确——抽象关注的是"隐藏复杂度、暴露接口"，而信息隐藏更具体地强调"内部数据结构不可见"。本题的核心问题是"直接访问内部数据"，违反的是信息隐藏而非抽象。
- C正确：信息隐藏——模块内部数据结构应对外不可见，必须通过接口访问。
- D错：局部化是"关系密切的元素放一起"，不是这里的越界访问问题。

信息隐藏的核心：模块只暴露"做什么"（接口），隐藏"怎么做"和"内部数据"。外部模块只能调接口，不能直接掏内部数据。这样内部实现变了，外部不受影响——这就是"低耦合"的底层支撑。抽象与信息隐藏常被混淆：抽象是"隐藏复杂度"，信息隐藏是"隐藏内部数据和实现"——前者关注接口设计，后者关注数据封装。

答案速查

题号	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
答案	B	B	D	A	D	C	C	B	B	C	B	C	C	A	C	A	C